Introduction to Data Analysis Using Pandas

Pandas

- is a Python library for data analysis
- has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008
- adds data structures and tools designed to work with table-like data (Series and DataFrame)
- provides tools for data manipulation: reshaping, merging, sorting, slicing, aggregation etc.
- allows handling missing data

Link: http://pandas.pydata.org/
Source code of Pandas: https://github.com/pandas-dev/pandas

Install Pandas

Install these libraries
py -m pip install numpy
py -m pip install pandas
py -m pip install matplotlib

Many popular Python toolboxes/libraries:

- NumPy: import numpy as np
- Pandas: import pandas as pd

Pandas is usually imported under the pd alias;
Then Pandas package can be referred to as pd instead of pandas

Pandas Series

Series

- A Pandas Series is like a column in a table.
- It is a one-dimensional array holding data of any type.

Labels in series

- If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.
- This label can be used to access a specified value.

Create a simple Pandas Series from a list:

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar)

print(myvar[0])
```

```
0    1
1    7
2    2
dtype: int64
1
```

Create Labels

With the index argument, you can name your own labels.
When you have created labels, you can access an item by referring to the label.

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)

print(myvar["y"])
```

```
x    1
y    7
z    2
dtype: int64
7
```

Pandas Series

Key/Value Objects as Series

- You can also use a key/value object, like a dictionary, when creating a Series.

Create a simple Pandas Series from a dictionary
The keys of the dictionary become the labels.

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

```
day1    420
day2    380
day3    390
dtype: int64
```

To select only some of the items in the dictionary, use the index argument and specify only the items you want to include in the Series.
Create a Series using only data from "day1" and "day2":

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories, index = ["day1", "day2"])

print(myvar)
```

```
day1    420
day2    380
dtype: int64
```

Pandas Series

- Similar to NumPy array, but it has more extra functionality. e.g. s.describe()
- All elements in NumPy array should be the same datatype, but Pandas Series doesn't have such requirement
- Similarities:
  - Accessing elements, 's[0]', 's[3:7]'
  - Looping 'for x in s'.
  - Convenient functions. 's.mean()', 's.max()'.
  - Vectorized operations. 's1 + s2' - Implemented in C, fast.

```
series1 = pd.Series(['a','b','c','d'])
series2 = pd.Series(['e','f','g','h'])
print("series1[0] = \n", series1[0])
print("series1[1:3] = \n", series1[1:3])
print("series1 + series2 = \n", series1+series2)
```

```
series1[0] =
 a
series1[1:3] =
 1    b
2    c
dtype: object
series1 + series2 =
 0    ae
1    bf
2    cg
3    dh
dtype: object
```

Series Labels

- The main difference between Pandas Series and NumPy arrays is that Pandas series has something called Labels.
- NumPy arrays are like souped version of Python lists.
- Similarly, a Pandas Series is like a cross between a list and a dictionary.
- List is a sequence of elements and you access each element or series by index.
- For dictionary, you access value by key.

Series Labels

- For Pandas Series, both index and key can become labels.
- If you didn't specify labels for a Pandas Series, then the values are labeled with their index number and 0,1,2,3... are used as labels by default.
- If you specified labels for a Pandas Series, Use **Series**.loc attribute to select some values from the given Series object based on the labels.

```
import pandas as pd

# Creating the Series
sr = pd.Series(['New York', 'Chicago', 'Toronto', 'Lisbon'])

# Creating the row axis labels
sr.index = ['City 1', 'City 2', 'City 3', 'City 4']

sr.loc[['City 4', 'City 3', 'City 1']]
# return
```

```
City 4      Lisbon
City 3     Toronto
City 1    New York
dtype: object
```

Series Labels

- Pandas **Series.iloc attribute** enables purely integer-location based indexing for selection by position over the given Series object.

- Use **Series**[] to access elements by **labels (either index or keys)**

```
import pandas as pd
sr = pd.Series(['New York', 'Chicago', 'Toronto', 'Lisbon'])
sr.index = ['City 1', 'City 2', 'City 3', 'City 4’]

print('''print: sr.loc[['City 1', 'City 2']]''')
print(sr.loc[['City 1', 'City 2’]])

print('''print: sr.iloc[0:2]''')
print(sr.iloc[0:2])

print('''print: sr[0:2]''')
print(sr[0:2])

print('''print: sr[['City 1', 'City 2']]''')
print(sr[['City 1', 'City 2']])
```

```
print: sr.loc[['City 1', 'City 2']]
City 1    New York
City 2     Chicago
dtype: object
print: sr.iloc[0:2]
City 1    New York
City 2     Chicago
dtype: object
print: sr[0:2]
City 1    New York
City 2     Chicago
dtype: object
print: sr[['City 1', 'City 2']]
City 1    New York
City 2     Chicago
dtype: object
```

Series Labels

```
series = pd.Series(['aa','bb','cc','dd'], index=['a','b','c','d'])
print("series.index = ", series.index)
print("access element by index, series.loc['a'] = ", series.loc['a'])
print("access element by position, series[2] = ", series[2])
print("access element by position, series.iloc[2] = ", series.iloc[2])
```

```
series.index =  Index(['a', 'b', 'c', 'd'], dtype='object')
access element by index, series.loc['a'] =  aa
access element by position, series[2] =  cc
access element by position, series.iloc[2] =  cc
```

Pandas DataFrame: Tabular data

- Pandas Series is like a column, a DataFrame is the whole table.
- A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.
- Pandas use the loc attribute to return one or more specified row(s)

Create a DataFrame from two Series

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```

For each series, the values may have different datatypes

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

```
#refer to the row index:
print(df.loc[0])
```

```
calories    420
duration     50
Name: 0, dtype: int64
```

This example returns a Pandas **Series**.

```
#use a list of indexes:
print(df.loc[[0, 1]])
```

```
   calories  duration
0       420        50
1       380        40
```

When using [], the result is a Pandas **DataFrame**.

Pandas DataFrame: Named Indexes

- With the index argument, you can name your own indexes.
- Use the named index in the loc attribute to return the specified row(s).

Add a list of names to give each row a name

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```

```
      calories  duration
day1       420        50
day2       380        40
day3       390        45
```

```
#refer to the named index:
print(df.loc["day2"])
```

```
calories    380
duration     40
Name: 0, dtype: int64
```

This example returns a Pandas **Series**.

```
#refer to the named indices:
print(df.loc[["day1","day2"]])
```

```
      calories  duration
day1       420        50
day2       380        40
```

When using [], the result is a Pandas **DataFrame**.

Pandas DataFrame: Tabular data

For each column, the values may have different datatypes

```
import pandas as pd
data = {"calories": [420, ‘unknown', 390],
        "duration": [50, 40, 45]}
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df.loc["day2"])
```

```
calories    unknown
duration         40
Name: day2, dtype: object
```

NumPy and Pandas for Tabular Data

- Two-dimensional data:
  - Python: list of lists
  - NumPy: 2D array
  - Pandas: DataFrame 2D arrays (**DataFrame** class), computationally efficient.
  - mean(), std(), etc operate on entire array

```
# reate a pandas 2d dataframe
df = pd.DataFrame(np.arange(6).reshape(2,3), index=['a','b'], columns=['c1','c2','c3'])
print('df.shape = ', df.shape)
print('df.index = ', df.index)
print('df = \n',df)
```

```
df.shape =  (2, 3)
df.index =  Index(['a', 'b'], dtype='object')
df =
    c1  c2  c3
a   0   1   2
b   3   4   5
```

Create Pandas DataFrame From Numpy Array, Python dict, and Python list

- Use Numpy array

```
# create a pandas 2d dataframe
df = pd.DataFrame(np.arange(6).reshape(2,3), index=['a','b'], columns=['c1','c2','c3'])
print('df.shape = ', df.shape)
print('df.index = ', df.index)
print('df = \n',df)
```

```
df.shape =  (2, 3)
df.index =  Index(['a', 'b'], dtype='object')
df =
    c1  c2  c3
a   0   1   2
b   3   4   5
```

- Use dict

```
# convert a dictionary to dataframe,
# the key is converted as column names.
dic = {'a':[1,2,3], 'b':[4,5,6]}
df = pd.DataFrame(dic, index=['x', 'y', 'z'])
print('df = \n', df)
print('df.mean = \n', df.mean())
print('df.values.mean = ',df.values.mean())
```

```
df =
    a  b
x  1  4
y  2  5
z  3  6
df.mean =
 a    2.0
b    5.0
dtype: float64
df.values.mean =  3.5
```

- Use List

```
# convert a 2D numpy array or list of lists to pandas dataframe
df_2 = pd.DataFrame([[0, 1, 2], [3, 4, 5]], columns=['A', 'B', 'C'])
print(df_2)
```

```
   A  B  C
0  0  1  2
1  3  4  5
```

Access Elements of DataFrame

- Access a ***single row***
  - df.loc[] to access rows by *rowName*
  - df.iloc[] to access rows by row*Index*
- Access a ***single element***.
  - df.loc[rowName, colName]
  - df.iloc[rowIndex, colName]
- Access ***columns*** using square brackets. df[colName]
- Use df.values() to get a 2D Numpy array. There are only values without column names and row indexes.

```
dic = {'a':[1,2,3], 'b':[4,5,6], 'c':[7,8,9]}
df = pd.DataFrame(dic, index=['x', 'y', 'z'])
print("Access elements by row, df.iloc[0] = ", df.iloc[0]) # access elements, first row
print("Each row in DataFrame is a Pandas Series: ", type(df.iloc[0])) # pandas series
print("Access elements by index, df.loc['x'] = ", df.loc['x']) # row whose index is 'x'
print("Access elements by column, df['a'] = ", df['a']) # column whose column name is 'a'
print("Access single element df.iloc[1, 2] = ", df.iloc[1, 2]) # # access 2nd row, 3rd column
print("Access multiple rows df.iloc[1:3] = ", df.iloc[1:3]) # access multiple rows
print("Access multiple columns, df[['a', 'b']]", df[['a', 'b']]) # access multiple columns
```

```
Access elements by row, df.iloc[0] =  a    1
b    4
c    7
Name: x, dtype: int64
Each row in DataFrame is a Pandas Series:  <class 'pandas.core.series.Series'>
Access elements by index, df.loc['x'] =  a    1
b    4
c    7
Name: x, dtype: int64
Access elements by column, df['a'] =  x    1
y    2
z    3
Name: a, dtype: int64
Access single element df.iloc[1, 2] =  8
Access multiple rows df.iloc[1:3] =     a  b  c
y  2  5  8
z  3  6  9
Access multiple columns, df[['a', 'b']]    a  b
x  1  4
y  2  5
z  3  6
```

Loading and writing data into a DataFrame

- A comma-separated values (CSV) file is a delimited text file that uses a comma to separate values. Each line of the file is a data record. Each record consists of one or more fields, separated by commas.
- Load data: pd.read_csv(filename)
- Write data to csv: pd.to_csv(filename)

```
dic = {'a':[1,2,3], 'b':[4,5,6], 'c':[7,8,9]}
df = pd.DataFrame(dic, index=['x', 'y', 'z'])
df.to_csv("test.csv", index=False) # write DataFrame to test.csv
df2 = pd.read_csv("test.csv")      # read DataFrame from disk
print(df2)
```

```
   a  b  c
0  1  4  7
1  2  5  8
2  3  6  9
```